COP3502: Computer Science 1
Final Exam Study Guide

Foundations and Algorithms

PUBLISHED BY GEMINI CLI

UCF COMPUTER SCIENCE 1

First printing, May 2026

Contents

Foundations

1. C Review & Dynamic Memory Allocation

1.1 Theoretical Core

Definition 1.1.1 — Pointers. A variable that stores a memory address as its value. Pointers allow direct manipulation of memory, efficient data sharing between functions (pass by reference), and referring to large data structures compactly. The `&` operator (address-of) retrieves the memory address of a variable, and the `*` operator (dereference) accesses or manipulates the value stored at that location.

Definition 1.1.2 — Structs. A custom data type defined using the `struct` keyword that allows grouping related variables (members/fields) of different data types into a single unit. It is often used alongside `typedef` to create aliases for easier reference.

Definition 1.1.3 — Dynamic Memory Allocation (DMA). Allocating memory at run-time from the heap space when the required size is unknown at compile time.

- `malloc()`: Allocates a specified number of bytes on the heap and returns a void pointer to it. The allocated memory is uninitialized.
- `calloc()`: Allocates memory for an array of elements and initializes all bytes to zero.
- `realloc()`: Resizes a previously allocated memory block, preserving its content up to the minimum of the old and new sizes.
- `free()`: Deallocates dynamically allocated memory, releasing it back to the heap to prevent memory leaks.

1.2 Algorithmic Tracing

■ **Example 1.1 — Dynamically Allocating a 2D Array.**

1. Determine the number of rows and columns needed.
2. Define a double pointer variable (e.g., `int **arr`).
3. Call `malloc()` to allocate an array of pointers (one for each row) and assign the result to the double pointer: `arr = malloc(rows * sizeof(int *));`.
4. Iterate over each row pointer (`i` from 0 to `rows - 1`).
5. Inside the loop, call `malloc()` to allocate memory for the columns and assign it to the current row pointer: `arr[i] = malloc(cols * sizeof(int));`.

■ **Example 1.2 — Freeing a 2D Array.**

1. Iterate over each row pointer (`i` from 0 to `rows - 1`).
2. Call `free()` on each row array: `free(arr[i]);`.
3. Finally, call `free()` on the array of pointers itself: `free(arr);`.

1.3 Code Implementation

The following snippet demonstrates dynamic memory allocation (including a 2D array) and the correct usage of struct member access operators.

```c
// Struct definitions demonstrating the dot (.) and arrow (->)
   operators
typedef struct Song_s {
    int song_id;
    char title[101];
} Song;

typedef struct Cat_s {
    char *name;          // Dynamically allocated string
    int age;
} Cat;

void allocate_and_access() {
    // Dot operator (.) usage with a struct instance
    Song s;
    s.song_id = 101;

    // Arrow operator (->) usage with a pointer to a struct
    Cat *cat = malloc(sizeof(Cat));
    cat->age = 5;

    // Remember to free dynamically allocated struct instances
    free(cat);
}

// 2D Array Dynamic Allocation & Deallocation
void manage_2d_array(int rows, int cols) {
    // 1. Allocation
    int **matrix = malloc(rows * sizeof(int *));
    for (int i = 0; i < rows; i++) {
        matrix[i] = malloc(cols * sizeof(int));
    }

    // 2. Deallocation
    for (int i = 0; i < rows; i++) {
        free(matrix[i]); // Free each row
    }
    free(matrix); // Free the array of pointers
}
```

1.4 Skills & Pitfalls

1.4.1 Must-Know Skills

- **Arrow vs. Dot Operator**: Use the dot operator (`.`) to access members of a struct variable (e.g., `list.count`). Use the arrow operator (`->`) to access members when you have a pointer to a struct (e.g., `list->count`), which is shorthand for `(*list).count`.
- **Pointer Arithmetic**: Understand the precedence of pointer arithmetic. `*(arr + i)` correctly dereferences the `i`-th element, whereas `*arr + i` dereferences the first element and

adds `i` to its value.

- **Constructors and Destructors**: Modularize code by defining helper functions to handle the allocation (e.g., `createStore`) and deallocation (e.g., `freeStore`) of complex dynamic structures to reduce the risk of memory leaks.

1.4.2 Common Mistakes (Pitfalls)

- **Memory Leaks**: Occur when dynamically allocated memory is no longer needed but is not released using `free()`. Every `malloc()` should have a corresponding `free()`.
- **Dangling Pointers**: Continuing to use a pointer after the memory it points to has been freed.
- **Segmentation Faults**: Run-time errors caused by memory access violations, such as dereferencing an invalid memory location or an uninitialized pointer. Always initialize pointers to `NULL` and check them defensively (e.g., `if (ptr != NULL)`) before dereferencing.

2. Recursion & Recursive Solutions

2.1 Theoretical Core Concepts

Definition 2.1.1 — Recursion. Defining a problem in terms of a simpler version of itself.

Definition 2.1.2 — Base Case (Stopping Condition). The condition where the problem is small enough to be solved directly. It is required to prevent infinite recursion.

Definition 2.1.3 — Recursive Step (Reduction Step). Breaking the big problem into smaller subproblems by having the function call itself.

- **Stack Frames**: Each function call creates its own copy of all local variables, conceptually

like stacking "Post-it notes". A return expression must be evaluated to a single value before control is passed back to the caller.

- **Tail Recursion vs. Head Recursion**: Tail recursion is where the recursive call is the last operation. Head recursion is where the recursive call happens before any other operations.

2.2 Algorithmic Tracing

■ **Example 2.1 — Tracing factorial(4).** Given the logic `return n * factorial(n - 1);` with base case `if (n <= 1) return 1;`

1. `main()` calls `factorial(4)`
2. Frame `f(4)`: evaluates `4 * f(3)`. Calls `f(3)`.
3. Frame `f(3)`: evaluates `3 * f(2)`. Calls `f(2)`.
4. Frame `f(2)`: evaluates `2 * f(1)`. Calls `f(1)`.
5. Frame `f(1)`: hits base case `n <= 1`, returns 1.
6. Frame `f(2)`: receives 1, evaluates `2 * 1 = 2`, returns 2.
7. Frame `f(3)`: receives 2, evaluates `3 * 2 = 6`, returns 6.
8. Frame `f(4)`: receives 6, evaluates `4 * 6 = 24`, returns 24 to `main()`.

2.3 Code Implementation

2.3.1 Towers of Hanoi

```
void towerOfHanoi(int n, char source, char dest, char aux) {
    if (n == 1) {
        printf("Move␣disk␣1␣from␣pole␣%c␣to␣pole␣%c\n", source,
            dest);
        return;
    }
    // Move n-1 disks from source to aux, using dest as auxiliary
    towerOfHanoi(n - 1, source, aux, dest);
    // Move the nth disk from source to dest
    printf("Move␣disk␣%d␣from␣pole␣%c␣to␣pole␣%c\n", n, source,
        dest);
    // Move the n-1 disks from aux to dest, using source as
        auxiliary
    towerOfHanoi(n - 1, aux, dest, source);
}
```

2.3.2 Permutations (Brute Force)

```
void permutation_fill(Song *solution, int size, int pos, const Song
    *actual, int *is_used) {
```

```
    // Base Case: If the solution array is completely filled
    if (pos == size) {
        permutation_print(solution, size);
        return;
    }

    // Recursive Case: Fill the current slot and recursively fill
       remaining slots
    for (int x = 0; x < size; x++) {
        if (is_used[x] == 0) {
            solution[pos] = actual[x];
            is_used[x] = 1; // Mark element as used
            permutation_fill(solution, size, pos + 1, actual,
               is_used);
            is_used[x] = 0; // Backtrack (unmark)
        }
    }
}
```

2.4 Skills & Pitfalls

2.4.1 Common Mistakes

- **Infinite Recursion / Stack Overflow**: Forgetting a base case or passing parameters that do not progress toward the base case, eventually resulting in a Segmentation Fault.
- **Incorrect Order of Operations**: Putting operations after the recursive call when they should be before it (or vice versa), breaking intended logic.

2.4.2 Must-Know Skills

- **Brute Force Enumeration Framework**: Using recursion to systematically check all candidates for a solution (e.g., subsets, permutations). A `pos` index tracks depth, and a loop handles the choices at each index.
- **Tracking State**: Maintaining an `is_used` array when generating permutations to ensure elements are not reused in the same candidate solution.
- **Understanding Overhead**: Recognizing that recursion tends to be less efficient than its iterative counterpart due to function call overhead. Some recursive algorithms (like basic Fibonacci) can do massive amounts of redundant work.

3. Algorithm Analysis & Algorithm Design

3.1 Theoretical Core: Big-O, Omega, and Theta

Definition 3.1.1 — Big-O Notation (O). Represents the asymptotic upper bound of an algorithm's runtime or space. It describes the worst-case scenario, guaranteeing the algorithm will take no more than a certain amount of time for sufficiently large inputs.

Definition 3.1.2 — Omega Notation (Ω). Represents the asymptotic lower bound. It describes the best-case scenario, guaranteeing the algorithm will take at least a certain amount of time.

Definition 3.1.3 — Theta Notation (Θ). Represents the asymptotically tight bound. It is used when an algorithm is bounded both above and below by the same function (i.e., both Big-O and Omega are the same).

3.2 Common Time Complexities

Theorem 3.2.1 — Hierarchy of Time Complexities. Common algorithm runtimes categorized from most efficient to least efficient:

- $O(1)$ **- Constant Time:** Runtime is independent of the input size. Example: Array index access.
- $O(\log n)$ **- Logarithmic Time:** Runtime grows logarithmically. Example: Binary search on a sorted list.
- $O(n)$ **- Linear Time:** Runtime grows proportionally with the input size. Example: Linear search through an array.
- $O(n \log n)$ **- Linearithmic Time:** Typical of optimal comparison-based sorts. Example: Merge sort or the binary search intersection strategy.
- $O(n^2)$ **- Quadratic Time:** Runtime grows with the square of the input size. Example: Comparing all pairs in an array or a nested loop linear search intersection strategy.

3.3 Analyzing Loops and Nested Loops

3.3.1 Single Loops

The runtime correlates to the number of iterations times the work done inside the loop. For n iterations doing constant work, the time is $O(n)$.

3.3.2 Nested Loops

Evaluated by multiplying the number of executions of the inner loop by the outer loop (when independent). For example, an outer loop running n times and an inner loop running m times results in $O(n \times m)$ time.

3.3.3 Algorithmic Tracing: Sorted Array Matching

When counting common elements in two sorted arrays of sizes n and m, different strategies yield vastly different runtimes:

■ **Example 3.1 — Strategy 1: Linear Search.** An outer loop over n elements and an inner loop over m elements yields $O(n \times m)$.

■ **Example 3.2 — Strategy 2: Binary Search.** A loop over n elements performing a binary search in m elements yields $O(n \log m)$.

■ **Example 3.3 — Strategy 3: Two Pointers.** Iterating both arrays simultaneously using two index variables yields an efficient $O(n+m)$.

3.4 Code Implementation

The Two-Pointer Strategy is the most efficient way to find the intersection of two sorted arrays.

Listing 3.1: Two-Pointer Strategy for Sorted Array Intersection

```c
#include <stdio.h>

// Returns the number of common elements between two sorted arrays
int count_common(int arr1[], int n, int arr2[], int m) {
    int i = 0, j = 0;
    int count = 0;

    // Advance the index of the smaller value, count when equal
    while (i < n && j < m) {
        if (arr1[i] == arr2[j]) {
            count++;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    return count;
}
```

3.5 Skills & Pitfalls

3.5.1 Must-Know Skills

- **Empirical Runtime Calculation:** Using experimental elapsed times to find the theoretical constant C. Given $T(n) = C \cdot f(n)$, we can calculate $C = T(n)/f(n)$. This enables the prediction of runtimes for arbitrary input sizes.
- **Solving Constraints:** Re-arranging empirical equations to find the maximum allowed input size N for a given constraint T.

3.5.2 Common Mistakes (Pitfalls)

R **Ignoring Constant Factors:** While $O(N)$ ignores constants theoretically, in practice (empirical runtime), the constant factor dictates performance for realistic input sizes.

R **Misidentifying the Bottleneck:** Failing to recognize which loop or operation dominates the overall runtime (e.g., misidentifying the bottleneck as a single $O(N)$ loop when a hidden $O(N^2)$ operation is executed).

R **Assuming Equal Lengths:** Assuming inputs (like two arrays) always have the same length N, ignoring edge case runtimes like $O(M \times N)$ or $O(M \log N)$.

4. Math Foundations: Summations & Recurrences

4.1 Theoretical Core & Formulas

4.1.1 Summations

The process of adding things together, often represented by the capital Greek letter Sigma (Σ). To perform Big-O analysis, we seek a **closed-form** mathematical expression of the summation.

Theorem 4.1.1 — Core Summation Formulas. The following closed-form expressions are essential for algorithm analysis:

- **Constant Series:** $\sum_{i=1}^{n} c = c \cdot n$
- **Arithmetic Series (Sum of i):** $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = O(n^2)$
- **Sum of Squares (Sum of i^2):** $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} = O(n^3)$
- **Sum of Cubes:** $\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4} = O(n^4)$
- **Geometric Series:** $\sum_{i=0}^{n-1} ar^i = \frac{a(r^n-1)}{r-1}$
- **Infinite Geometric Series (where $|x| < 1$):** $\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$

4.1.2 Recurrences

A recurrence relation models the running time $T(n)$ of a recursive function by describing how to calculate a term based on previous terms in the sequence.

Definition 4.1.1 — Divide and Conquer General Form. $T(n) = aT(n/b) + D(n) + C(n)$

- a is the number of subproblems.
- b is the factor by which the input size is divided.
- $D(n)$ is the divide time, and $C(n)$ is the combine time.

4.2 Techniques

4.2.1 Master Theorem

Theorem 4.2.1 — Master Theorem. Used to quickly determine the Big-O running time of divide-and-conquer algorithms matching the form $T(n) = aT(n/b) + f(n)$. It compares the work done in splitting/combining $f(n)$ to the work done in the recursive calls $O(n^{\log_b a})$.

4.2.2 Iteration Method (Substitution Method)

A step-by-step technique to unroll a recurrence relation into a summation, which can then be reduced to a closed form.

1. **Expand** the recurrence step by step for at least 3 iterations.
2. **Notice the pattern** that emerges and write a generalized equation in terms of k.
3. **Determine the base case** to find the value of k that terminates the recurrence.
4. **Derive a closed-form solution** by substituting k back into the generalized equation and simplifying any summations.

4.3 Algorithmic Tracing: Iteration Method Example

■ **Example 4.1 — Solving $T(n) = 2T(n-1)+5$.** **Problem:** Solve $T(n) = 2T(n-1)+5$, with base case $T(1) = 1$.

Step 1: Expand (Unroll)

- $k = 1 : T(n) = 2T(n-1)+5$
- $k = 2 : T(n) = 2[2T(n-2)+5]+5 = 2^2T(n-2)+2(5)+5$
- $k = 3 : T(n) = 2^2[2T(n-3)+5]+2(5)+5 = 2^3T(n-3)+2^2(5)+2(5)+5$

Step 2: Generalize Pattern

- $T(n) = 2^kT(n-k)+\sum_{i=0}^{k-1} 5\cdot 2^i$

Step 3: Determine Base Case

- We hit the base case when $n-k = 1$.
- Solving for k, we get $k = n-1$.

Step 4: Substitute and Solve (Closed-Form)

- Substitute $k = n-1$ into the generalized equation: $T(n) = 2^{n-1}T(1)+5\sum_{i=0}^{(n-1)-1} 2^i$
- Knowing $T(1) = 1$, we get: $T(n) = 2^{n-1}(1)+5\sum_{i=0}^{n-2} 2^i$
- Apply the Geometric Series formula to the summation: $T(n) = 2^{n-1}+5\left(\frac{2^{n-1}-1}{2-1}\right)$
- Simplify: $T(n) = 2^{n-1}+5(2^{n-1}-1) = 6(2^{n-1})-5 = 3\cdot 2^n-5$

Result: $O(2^n)$

4.4 Skills & Pitfalls

R **Off-by-one errors in summation indices:** Known closed-form formulas usually assume the index starts at 1 (or 0 for geometric series). If a summation begins at a different value (e.g., $\sum_{i=10}^{n} i$), you must shift the summation or subtract the missing prefix before applying the formula.

R **Misapplying the Master Theorem cases:** Ensure the recurrence exactly matches the required form $T(n) = aT(n/b)+f(n)$ and carefully determine whether $f(n)$ is polynomially smaller, larger, or equal to $O(n^{\log_b a})$.

4.4.1 Must-Know Skills

- **Index Manipulation:** The ability to algebraically manipulate limits or split summations is critical.

- **Logarithm Properties:** When evaluating base cases for divide and conquer algorithms (e.g., $n/2^k = 1 \implies k = \log_2 n$), exponent and logarithm properties are heavily utilized.

II Data Structures

5. Lists, Stacks, and Queues

5.1 Theoretical Core

Definition 5.1.1 — Linked List. A collection of sequential nodes (linear). Each node contains data and a reference (pointer) to the next node. Only the last node does not have a reference to the next node (i.e., `NULL`). The list maintains a reference to the `head` (first node).

- **Singly Linked List**: Standard linear collection where each node points to the next.
- **Doubly Linked List**: Nodes keep track of three fields: `data`, `next`, and `prev`. A node knows both which node comes after it and which node comes before it.
- **Circular Linked List**: Keeping track of the `tail` is sufficient, as accessing the `head` simply requires accessing the `next` field of the `tail`.

Array vs. Linked Implementation:

- **Arrays** store data contiguously and can be dynamically allocated. However, moving elements around (shifting) for insertion or removal in the middle is an expensive operation. Arrays can also waste memory if allocated with a capacity significantly larger than their logical size.
- **Linked Lists** dynamically allocate memory per node, providing flexibility. They avoid the overhead of shifting elements during insertions and deletions, but they require

sequential traversal to access arbitrary elements.

Definition 5.1.2 — Queue. An Abstract Data Type (ADT) representing a collection of items following the **First In, First Out (FIFO)** principle. Implementation requires tracking both the `front` (first element to be removed) and the `rear` (last element added).

Definition 5.1.3 — Stack. An Abstract Data Type (ADT) representing a collection of elements following the **Last In, First Out (LIFO)** principle. Implementation requires tracking only the `topmost` element of the stack.

5.2 Algorithmic Tracing & Code Implementation

5.2.1 Linked Lists: Traversal & Generic Deletion

Listing 5.1: Linked List Traversal and Deletion

```
typedef struct SLLNode_s {
    int data;
    struct SLLNode_s *next;
} SLLNode;

typedef struct SLList_s {
    SLLNode *head;
} SLList;

// Traversal pattern
void sll_print_list(SLList *list) {
    SLLNode *ptr = list->head;
    while(ptr != NULL) {
        printf("%d␣", ptr->data);
        ptr = ptr->next;
    }
}

// Generic node deletion utilizing a trailing pointer
SLLNode* remove_element(SLList *list, int target) {
    SLLNode *ptr = list->head;
    SLLNode *trail = NULL;

    while (ptr != NULL && ptr->data != target) {
        trail = ptr;
        ptr = ptr->next;
```

```
    }

    if (ptr == NULL) return NULL; // Target not found

    if (trail == NULL) {
        list->head = ptr->next;   // Target is at the head
    } else {
        trail->next = ptr->next;  // Target is in the middle or end
    }

    ptr->next = NULL;
    return ptr; // Caller is responsible for free()
}
```

5.2.2 Stacks: Push and Pop

Listing 5.2: Stack Implementation (Linked List)

```
typedef struct Stack_s {
    SLLNode *top;
} Stack;

void push(Stack *s, int val) {
    SLLNode *newNode = malloc(sizeof(SLLNode));
    newNode->data = val;
    newNode->next = s->top;
    s->top = newNode;
}

int pop(Stack *s) {
    if (s->top == NULL) return -1; // Stack is empty

    SLLNode *tmp = s->top;
    int val = tmp->data;
    s->top = s->top->next;

    free(tmp);
    return val;
}
```

5.2.3 Queues: Enqueue and Dequeue

Listing 5.3: Queue Implementation (Linked List)

```
typedef struct Queue_s {
    SLLNode *front;
    SLLNode *rear;
} Queue;

void enqueue(Queue *q, int val) {
    SLLNode *newNode = malloc(sizeof(SLLNode));
    newNode->data = val;
    newNode->next = NULL;

```

```
    if (q->rear == NULL) {
        q->front = q->rear = newNode;
        return;
    }

    q->rear->next = newNode;
    q->rear = newNode;
}

int dequeue(Queue *q) {
    if (q->front == NULL) return -1; // Queue is empty

    SLLNode *tmp = q->front;
    int val = tmp->data;
    q->front = q->front->next;

    if (q->front == NULL) {
        q->rear = NULL; // Queue became empty
    }

    free(tmp);
    return val;
}
```

5.3 Skills & Pitfalls

Common Mistakes:

- **Losing the head pointer:** Never modify the `head` pointer directly when traversing the list. Always use a temporary pointer to iterate through nodes.
- **Ignoring NULL checks:** Failing to check if `ptr != NULL` before accessing `ptr->next` will lead to segmentation faults.
- **Memory Leaks:** Forgetting to store the reference to the next node before deallocating the current node.

Exercise 5.1 — Must-Know Skills. Master the use of a **trailing pointer** (`trail`) that stays one step behind the main traversal pointer (`ptr`). This is crucial for generic insertion and deletion operations. ■

6. Trees

6.1 Binary Search Trees (BST)

Definition 6.1.1 — Binary Search Tree. A hierarchical, node-based structure where for any given node y:

- All nodes in its left subtree have values less than or equal to $y.data$.
- All nodes in its right subtree have values strictly greater than $y.data$.

Deletion Scenarios:

1. **No Children (Leaf)**: Remove the node and set the parent's pointer to `NULL`.
2. **One Child**: "Promote" the target's single child to take its place.
3. **Two Children**: Find the **successor** (min in right subtree) or **predecessor** (max in left subtree). Overwrite target data and recursively delete the successor/predecessor.

Listing 6.1: Recursive BST Insertion

```
BSTNode* insertRec(BSTNode* root, int value) {
    if (root == NULL) {
        BSTNode* newNode = (BSTNode*)malloc(sizeof(BSTNode));
        newNode->data = value;
```

```
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }

    if (value < root->data) {
        root->left = insertRec(root->left, value);
    } else if (value > root->data) {
        root->right = insertRec(root->right, value);
    }

    return root;
}
```

6.2 AVL Trees

Definition 6.2.1 — AVL Tree. A self-balancing BST that ensures the tree's height remains $O(\log n)$. It uses a **Balance Factor (BF)**: $Height_{Left} - Height_{Right}$. Valid BFs are -1, 0, and +1.

Definition 6.2.2 — AVL Rotation Cases. When a node's BF reaches ± 2, rotations are applied based on the configuration:

- **LL (Left-Left)**: Pivot BF = +2, Left Child BF = +1 (or 0). Fix: **Single Right Rotation**.
- **RR (Right-Right)**: Pivot BF = -2, Right Child BF = -1 (or 0). Fix: **Single Left Rotation**.
- **LR (Left-Right)**: Pivot BF = +2, Left Child BF = -1. Fix: **Left Rotation at child**, then **Right Rotation at pivot**.
- **RL (Right-Left)**: Pivot BF = -2, Right Child BF = +1. Fix: **Right Rotation at child**, then **Left Rotation at pivot**.

■ **Example 6.1 — RL Double Rotation Trace.** Inserting **10, 30, 20**:

1. **10** is root.
2. **30** added as right child of 10.
3. **20** added as left child of 30.
4. Pivot 10 has BF = -2. Right child 30 has BF = +1. Configuration is **RL**.
5. **Step 1**: Right Rotate at 30. 20 moves up, 30 becomes its right child.
6. **Step 2**: Left Rotate at 10. 20 becomes root, 10 is left child, 30 is right child.

Listing 6.2: AVL Single Rotations

```
// Single Right Rotation (Fixes LL)
AVLNode* rightRotate(AVLNode* y) {
    AVLNode* x = y->left;
    AVLNode* T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
    return x;
}

// Single Left Rotation (Fixes RR)
AVLNode* leftRotate(AVLNode* x) {
    AVLNode* y = x->right;
    AVLNode* T2 = y->left;
    y->left = x;
    x->right = T2;
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
    return y;
}
```

6.3 Skills & Pitfalls

R **Height Updates**: After every modification (insertion/deletion) and rotation, heights must be updated bottom-up. Failing to do so breaks the balance factor calculations.

Exercise 6.1 — Deletion Cascades. While an insertion causes at most one imbalance that needs fixing, a **deletion** can cause cascading imbalances all the way up the tree. You must check every ancestor after a deletion. ■

7. Heaps

7.1 Theoretical Core

Definition 7.1.1 — Binary Heap. A complete binary tree that strictly follows a specific heap property. A complete binary tree has all levels fully populated except possibly the last, where leaves are placed as far left as possible.

Definition 7.1.2 — Min-Heap Property. The value of an arbitrary node must be less than or equal to the value of its children. The root inherently contains the smallest element in the tree.

Definition 7.1.3 — Max-Heap Property. The value of an arbitrary node must be greater than or equal to the value of its children. The root contains the largest element.

7.1.1 Array Representation (0-Indexed)

Binary heaps are efficiently implemented using 0-indexed arrays without using linked pointers. Given a node at index i:

- **Index of Left Child:** $2i+1$
- **Index of Right Child:** $2i+2$
- **Index of Parent:** $(i-1)/2$ (integer division)

R Child node indices must be strictly validated as $< size$, while parent indices must be ≥ 0.

7.1.2 Key Operations & Time Complexity

- **Heapify / Percolate Down:** Pushes a node down the tree to its correct position. Time Complexity: $O(\log n)$.
- **Percolate Up / Heapify Up:** Pushes a node up the tree. Used during insertion. Time Complexity: $O(\log n)$.
- **Build-Heap (Floyd's Method):** A bottom-up approach to build a heap from an unsorted array. Time Complexity: $O(n)$.
- **Heap Insertion:** Place at the end and percolate up. Time Complexity: $O(\log n)$.

7.2 Mathematical Proof: O(n) Build-Heap

Theorem 7.2.1 — O(n) Build-Heap Complexity. The `BuildHeap` operation takes $O(n)$ time in the worst case. This is proven by summing the effort of calling `Heapify` on all internal nodes.

Proof. Let n be the number of nodes and $h = \lfloor \lg n \rfloor$ be the height. At level i, there are 2^i nodes. The effort for `Heapify` at level i is $h-i$. The total effort S is:

$$S = \sum_{i=0}^{h-1} 2^i (h-i)$$

Derivation: Expand S:

$$S = 2^0(h) + 2^1(h-1) + 2^2(h-2) + \cdots + 2^{h-1}(1)$$

Multiply by 2:

$$2S = 2^1(h) + 2^2(h-1) + 2^3(h-2) + \cdots + 2^h(1)$$

Subtract S from $2S$:

$$S = -h + 2^1 + 2^2 + \cdots + 2^h = -h + \sum_{i=1}^{h} 2^i$$

Using the geometric series formula $\sum_{i=0}^{h} 2^i = 2^{h+1} - 1$:

$$S = -h + (2^{h+1} - 1) - 2^0 = 2^{h+1} - h - 2$$

Since $2^h \approx n$:

$$S \approx 2n - \lg n - 2$$

Thus, the operation is $O(n)$. ■

7.3 Algorithmic Tracing

■ **Example 7.1 — Floyd's Method Trace.** Unsorted array: $[4,1,9,2,16,5,10,14,3,7]$, $size = 10$.

1. **Last internal node:** Index $(9-1)/2 = 4$ (value 16).
2. **i = 4:** Swap 16 with child 7. Array: $[4,1,9,2,7,5,10,14,3,16]$.
3. **i = 3:** Value 2. Minimum. No swap.
4. **i = 2:** Swap 9 with child 5. Array: $[4,1,5,2,7,9,10,14,3,16]$.
5. **i = 1:** Value 1. Minimum. No swap.
6. **i = 0:** Swap 4 with child 1, then with 2, then with 3.
7. **Final Array:** $[1,2,5,3,7,9,10,14,4,16]$.

7.4 Code Implementation

```c
// Heapify Down for a Min-Heap
void percolateDown(int *arr, int size, int idx) {
    int min = idx;
    int left_idx = 2 * idx + 1;
    int right_idx = 2 * idx + 2;

    if (left_idx < size && arr[left_idx] < arr[min]) {
        min = left_idx;
    }
    if (right_idx < size && arr[right_idx] < arr[min]) {
        min = right_idx;
    }

    if (min != idx) {
        int tmp = arr[idx];
        arr[idx] = arr[min];
        arr[min] = tmp;
        percolateDown(arr, size, min);
    }
}

// Insertion into a Min-Heap
void insert(int *arr, int *size, int capacity, int val) {
    if (*size >= capacity) return;

    int idx = *size;
    arr[idx] = val;
    (*size)++;

    int parent_idx = (idx - 1) / 2;
    while (idx > 0 && arr[idx] < arr[parent_idx]) {
        int tmp = arr[idx];
```

```
33          arr[idx] = arr[parent_idx];
34          arr[parent_idx] = tmp;
35
36          idx = parent_idx;
37          parent_idx = (idx - 1) / 2;
38      }
39  }
```

7.5 Skills & Pitfalls

7.5.1 Must-Know Skills

- **Heapify Selection:** Use `Percolate Up` for insertions and `Percolate Down` for deletions and $O(n)$ build-heap.
- **Array Mapping:** Fluidly convert between tree visualization and array indices.

7.5.2 Common Mistakes (Pitfalls)

- **Index Math:** Using $i/2$ instead of $(i-1)/2$ in 0-indexed heaps leads to incorrect parent pointers.
- **Bounds Checking:** Always verify $2i+1 < size$ and $2i+2 < size$ before accessing children.
- **Memory Management:** Ensure the array has sufficient capacity before performing insertions to avoid buffer overflows.

8. Tries and Hashing

8.1 Theoretical Core

8.1.1 Tries

Definition 8.1.1 — Trie. A multiway tree (often called a prefix tree) used to store a list of strings. Nodes represent characters (often implied by the index in a child pointer array), and the path from the root to a node defines a prefix or word.

- **Node Structure:** Contains an array of 26 pointers (for 'a'-'z') and a boolean flag `is_end_of_word`.
- **Complexity:** $O(k)$ for Insert, Search, and Remove, where k is the length of the string.
- **Space Trade-off:** Fast prefix searches at the cost of high memory usage due to many NULL pointers.

8.1.2 Hash Tables

Definition 8.1.2 — Hash Table. An implementation of the Table ADT that stores (key, value) pairs and allows for $O(1)$ expected time retrieval using a hash function.

- **Hash Function ($h(k)$):** Maps a key k to a slot index. Common method: $h(k) = k \pmod{m}$.

- **Load Factor (α):** Ratio of elements n to slots m ($\alpha = n/m$).
- **Collision Resolution:**
  - **Separate Chaining:** Each slot points to a linked list.
  - **Open Addressing:** Probe for the next available slot.

8.2 Algorithmic Tracing

■ **Example 8.1 — Hashing with Probing.** Table size $m = 7$, $h(k) = k \pmod 7$. Insert keys: $A(6), B(6), C(5), D(5)$.

Linear Probing ($h(k,i) = (h'(k) + i) \pmod 7$):

1. A: $6 \pmod 7 = 6$. Insert at index 6.
2. B: $6 \pmod 7 = 6$ (Coll). Probe $i = 1$: $(6+1) \pmod 7 = 0$. Insert at 0.
3. C: $5 \pmod 7 = 5$. Insert at index 5.
4. D: $5 \pmod 7 = 5$ (Coll). Probe $i = 1$: $(5+1) \pmod 7 = 6$ (Coll). Probe $i = 2, 3$: Insert at 1.

Quadratic Probing ($h(k,i) = (h'(k) + i^2) \pmod 7$):

1. A, B, C: Same as linear (at indices 6, 0, 5).
2. D: $5 \pmod 7 = 5$ (Coll). Probe $i = 1$: $(5+1^2) \pmod 7 = 6$ (Coll). Probe $i = 2$: $(5+2^2) \pmod 7 = 2$. Insert at 2.

8.3 Code Implementation

```c
#define MAX_CHILD_COUNT 26

typedef struct TrieNode_s {
    struct TrieNode_s *children[MAX_CHILD_COUNT];
    int is_end_of_word;
} TrieNode;

// Simple String Hash Function
int string_to_int(char *str) {
    int sum = 0;
    for (int i = 0; str[i] != '\0'; i++) {
        sum += str[i];
    }
    return sum;
}

int hash_function(int key, int m) {
    return key % m;
}
```

8.4 Skills & Pitfalls

8.4.1 Must-Know Skills

- **Prefix Matching:** Tries are superior for finding all words starting with "pre" by traversing to the "e" node and exploring the subtree.
- **Table Sizing:** For Quadratic Probing, ensure the table size m is a prime number and the load factor $\alpha < 0.5$ to guarantee finding an empty slot.

8.4.2 Common Mistakes (Pitfalls)

- **Clustering:** Linear probing suffers from **primary clustering**, where long runs of occupied slots increase search time. Quadratic probing mitigates this but can cause **secondary clustering**.
- **Tombstones:** When deleting from an open-addressing table, use a `DELETED` marker. Clearing the slot to `NULL` breaks the probe sequence for other keys.
- **Trie Memory Management:** Always use a post-order traversal to `free()` trie nodes to avoid massive memory leaks.

III

Algorithms & Prep

9. Sorting Algorithms

9.1 Theoretical Core

Sorting is a fundamental operation in computer science, used to arrange data in a specific order (ascending or descending). Algorithms are generally categorized by their approach: elementary or divide and conquer.

Definition 9.1.1 — Elementary Sorting. Algorithms that typically use nested loops and have $O(n^2)$ average-case time complexity.

- **Selection Sort:** Repeatedly finds the minimum element from the unsorted region and swaps it into the sorted region.
- **Insertion Sort:** Builds the sorted list one element at a time by inserting each new element into its correct position.
- **Bubble Sort:** Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

Definition 9.1.2 — Divide and Conquer Sorting. Algorithms that recursively break a problem into sub-problems until they are simple enough to solve directly.

- **Merge Sort:** Recursively divides the array in half, sorts the halves, and merges them.
- **Quick Sort:** Partitions the array around a pivot element and recursively sorts the resulting partitions.

9.2 Time & Space Complexities

Theorem 9.2.1 — Sorting Complexity Bounds. The following table summarizes the performance characteristics of standard sorting algorithms.

Algorithm	Best	Average	Worst	Space	Stable?
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No

Table 9.1: Comparison of Sorting Algorithms

R Merge Sort's $O(n)$ space complexity is due to the auxiliary array needed for the merge step. Quick Sort's space complexity is $O(\log n)$ on average due to the recursion stack.

9.3 Algorithmic Tracing

■ **Example 9.1 — Merge Sort Trace.** Tracing `[38, 27, 43, 3, 9, 82, 10]`:

1. **Divide:** `[38, 27, 43, 3]` and `[9, 82, 10]`
2. **Divide:** `[38, 27]`, `[43, 3]`, `[9, 82]`, `[10]`
3. **Merge:** `[27, 38]` and `[3, 43]` $\rightarrow$ `[3, 27, 38, 43]`
4. **Merge:** `[9, 82]` and `[10]` $\rightarrow$ `[9, 10, 82]`
5. **Final Merge:** $\rightarrow$ `[3, 9, 10, 27, 38, 43, 82]`

■ **Example 9.2 — Quick Sort Partition Trace.** Target Array: `[8, 3, 1, 7, 0, 10, 2]`, Pivot: 2.

- **Initial:** `i` tracks smaller elements. `j` scans the array.
- `j=2 (val 1)`: $1 \leq 2$, swap with `arr[0]`. Array: `[1, 3, 8, 7, 0, 10, 2]`
- `j=4 (val 0)`: $0 \leq 2$, swap with `arr[1]`. Array: `[1, 0, 8, 7, 3, 10, 2]`
- **Final Swap:** Swap pivot (2) with `arr[2]`.
- **Result:** `[1, 0, 2, 7, 3, 10, 8]`

9.4 Code Implementation

9.4.1 Merge Step

Listing 9.1: Merge Step Implementation

```
void merge(int arr[], int low, int mid, int high) {
    int size = high - low + 1;
    int *tmp = malloc(size * sizeof(int));
    int i = low, j = mid, p = 0;

    while (i < mid && j <= high) {
        if (arr[i] <= arr[j]) tmp[p++] = arr[i++];
        else tmp[p++] = arr[j++];
    }
    while (i < mid) tmp[p++] = arr[i++];
    while (j <= high) tmp[p++] = arr[j++];

    for (int x = low, pos = 0; x <= high; x++, pos++) {
        arr[x] = tmp[pos];
    }
    free(tmp);
}
```

9.4.2 Quick Sort Partition

Listing 9.2: Quick Sort Partition Step

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Last element as pivot
    int j = low - 1;

    for (int i = low; i <= high - 1; i++) {
        if (arr[i] <= pivot) { // HIGHLIGHT: Core Partition Logic
            j++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
```

```
    }
    // Swap pivot to its final position
    int temp = arr[j + 1];
    arr[j + 1] = arr[high];
    arr[high] = temp;

    return j + 1;
}
```

9.5 Skills & Pitfalls

R A major pitfall in Quick Sort is the $O(n^2)$ worst-case, which occurs on already sorted arrays when the pivot is consistently the maximum or minimum element. Using a random pivot or "median of three" is a critical skill for optimization.

Exercise 9.1 — Complexity Analysis. Why does Merge Sort require $O(n)$ space while Selection Sort only requires $O(1)$? Explain in terms of auxiliary data structures. ■

10. Bitwise Operators & Binary Numbers

10.1 Theoretical Core

Understanding the binary representation of data is crucial for low-level optimization and manipulating hardware flags.

Definition 10.1.1 — Binary Systems.

- **Binary (Base-2):** A system using digits 0 and 1.
- **Two's Complement:** The standard for representing signed integers. To negate a number: invert all bits and add 1.
- **MSB & LSB:** The Most Significant Bit (sign bit in signed types) and Least Significant Bit (rightmost bit).

10.1.1 Core Bitwise Operators

Theorem 10.1.1 — Bitwise Operations. The following operators allow for bit-level manipulation:

- `&` (AND): 1 if both bits are 1.
- `|` (OR): 1 if either bit is 1.
- `^` (XOR): 1 if bits are different.
- `~` (NOT): Inverts all bits.
- `«` (Left Shift): Multiplies by 2^n.
- `»` (Right Shift): Divides by 2^n.

10.2 Algorithmic Tracing

■ **Example 10.1 — Bit Masking Tricks.** Using a `MASK` (e.g., `1 « n` for the n-th bit):

- **Set Bit On:** `flags |= MASK;`
- **Set Bit Off:** `flags &= ~MASK;`
- **Toggle Bit:** `flags ^= MASK;`
- **Check Bit:** `if (flags & MASK)`

■ **Example 10.2 — Power of Two Check.** A clever use of bitwise logic to check if n is a power of 2: `(n & (n - 1)) == 0` If $n = 8(1000_2)$, then $n-1 = 7(0111_2)$. Their AND is 0.

10.3 Code Implementation

Listing 10.1: Common Bitwise Functions

```
// Check if a number is odd
int is_odd(int num) {
    return num & 1;
}

// Find value of rightmost 1 bit
int rightmost_one_bit_value(int N) {
```

```
    int m = 1;
    while ((N & m) == 0) {
        m = m << 1;
    }
    return m;
}

// Bitmask demo
void apply_mask() {
    unsigned int flags = 0;
    unsigned int MASK = 1 << 3; // 4th bit
    flags |= MASK;  // Set on
    flags &= ~MASK; // Set off
}
```

10.4 Skills & Pitfalls

Operator Precedence: Bitwise operators (`&`, `|`, `^`) have lower precedence than relational operators (`==`, `!=`). Always use parentheses: `(x & mask) == mask`.

Sign Extension: Right shifting signed negative integers (`int`) often results in an arithmetic shift (filling with 1s), while unsigned types (`unsigned int`) use a logical shift (filling with 0s). Prefer `unsigned` types for bit manipulation.

Exercise 10.1 — Base Conversion. Convert 41_{10} to binary. Then, convert the binary result to hexadecimal. ■

11. Exam Prep & Programming Assignments

11.1 High-Frequency Exam Topics

Success in CS1 and the Foundation Exam requires mastery of both theoretical analysis and applied data structure operations.

Theorem 11.1.1 — Algorithmic Analysis.

- **Recurrence Relations:** Mastery of the Master Theorem and the Iteration Technique is mandatory.
- **Summations:** Algebraic manipulation of closed-form solutions.
- **Runtime Analysis:** Determining Big-O for nested loops and recursive calls.

Definition 11.1.1 — Core Data Structure Operations.

- **Binary Heaps:** Tracing insertions and deletions in an array-based heap.
- **BST & Tries:** Implementing recursion for counting or searching.
- **Linked Lists:** Pointer manipulation for insertion, deletion, and reordering.

11.2 Programming Assignment Core Logic

Each Programming Assignment (PA) introduces a specific algorithmic "trick" that builds on standard data structures.

■ **Example 11.1 — PA Hybrid Sorting.** In PA4, Merge Sort and Quick Sort were optimized by switching to **Insertion Sort** once the subarray size fell below a certain threshold (e.g., 30). This leverages the efficiency of $O(n^2)$ algorithms on small datasets.

■ **Example 11.2 — PA Order Statistic Tree.** In PA5, a standard BST was augmented with a `subtree_size` field at each node. This allowed for $O(h)$ time complexity when searching for the k**-th smallest element**.

■ **Example 11.3 — PA Mode-Dependent Heap.** In PA6, a Binary Heap was implemented with a mode-dependent comparator, allowing it to toggle between a **Min-Heap** (Triage Mode) and a **Max-Heap** (Adoption Mode) using a bottom-up $O(n)$ `heapify`.

11.3 Foundation Exam Core Patterns

R [The Recursive Traversal Pattern] Most tree and graph problems follow a standard template:

1. **Base Case:** Check for `NULL` or terminal states.
2. **Recursive Step:** Aggregate results from child nodes.
3. **Return:** Pass the result back up the stack.

R [Pointer Safety] In Linked List and DMA questions, always trace your pointers. Ensure you don't "drop" the rest of the list when reassigning `next` pointers.

11.4 Practice Problems

Exercise 11.1 — Recurrence Iteration. Solve the recurrence $T(n) = 2T(n/2) + n$ using the iteration technique. Show each step of the expansion. ■

Exercise 11.2 — Heap Trace. Given an empty min-heap, trace the insertion of the following values: `[10, 5, 12, 3, 8]`. Draw the resulting array representation. ■